

claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_133acdee-dab\agent-aac1cbd8e38e723b5.jsonl`  
- SHA-256: `cacd2bb97e9cf11712e15925561639776aca18bc6764683bf2d4db4075033ab8`  
- Source modified: `2026-07-06T17:31:10+00:00`  
- Imported at: `2026-07-08T15:59:35+00:00`  
- project: `wf_133acdee-dab`  
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`
```

Transcript

1. user (2026-07-06T17:30:35.319Z)

You are an ADVERSARIAL VERIFIER. Try HARD to REFUTE the following review finding by reading the ACTUAL code in C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (and the diff at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff). Default to REFUTED if you cannot concretely reproduce the defect by pointing at real code lines. A finding is CONFIRMED only if you can trace a concrete input/state that produces the claimed wrong behavior in the code AS WRITTEN (not as you imagine it). If the claimed line/behavior does not exist, or a guard/caller already handles it, or it is out-of-scope speculation, mark REFUTED. If genuinely unclear after reading, mark UNCERTAIN.

```
FINDING (from the conventions-tests lens):  
severity=low | tests/test_worker.py:316  
summary: test_cmd_infer_validation_failure_returns_2 exercises the new _fail(2) report-writing path but asserts only rc==2, leaving the newly-added validation-failure report.json (status/segment_count) unlocked by any assertion.  
detail: _fail now writes+pushes a status="failed" report.json for EVERY failure rc, including validation (rc 2) and unknown-segmenter (rc 2) ? worker/__main__.py line 283-294. Only the rc-3 detector-failure test asserts report.json content and the done-marker; the rc-2 validation test (test_worker.py line 316-341) checks nothing about the written report and passes no --done-uri, so neither the report status nor the marker is verified for the validation branch. A regression that wrote status="success" (or skipped the artifact) on validation failure would not be caught.  
failure_scenario: A refactor that mis-passed the status literal (or reordered so validation _fail skipped _serialize_and_push_outputs) would leave a validation-failed video with a status="success" or missing report.json; the suite stays green because the only validation test asserts rc alone.  
suggested_fix: Extend test_cmd_infer_validation_failure_returns_2 to read outputs/<video_id>/report.json and assert status=="failed" and segment_count==0 (and, with a --done-uri, marker returncode==2/status=="failed"), mirroring the rc-3 test.
```

Verify against the real code and return your verdict.

2. assistant (2026-07-06T17:30:41.669Z)

I'll verify this finding by reading the actual test code and the worker implementation.

3. user (2026-07-06T17:30:43.186Z)

```
300             "--done-uri",  
301             str(done),  
302         ]  
303     )  
304     # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).  
305     assert rc == 3
```

```

306     # report.json is written as an explicit FAILURE, not a false empty "success".
307     report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
308     assert report["status"] == "failed" and report["segment_count"] == 0
309     assert report["segments"] == []
310     # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
311     marker = json.loads(done.read_text())
312     assert marker["returncode"] == 3 and marker["status"] == "failed"
313     assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
314
315
316 def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
317     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
318     monkeypatch.setattr(
319         wmain.validator_registry,
320         "run_all_with_context",
321         lambda path, cfg: ([_FailResult()], {}),
322     )
323     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
324     video = tmp_path / "in.mp4"
325     video.write_bytes(b"X")
326     cfg = tmp_path / "config.json"
327     cfg.write_text("{}")
328     rc = wmain.main(
329         [
330             "infer",
331             "--video-uri",
332             str(video),
333             "--out-uri",
334             str(tmp_path / "o"),
335             "--config",
336             str(cfg),
337             "--scratch",
338             str(tmp_path / "s"),
339         ]
340     )
341     assert rc == 2
342
343
344 # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---
345
346
347 @pytest.mark.parametrize(
348     "fname,expected",
349     [
350         (
351             "2026-06-22_GX020094.rallies.csv",
352             "GX020094",
353         ), # canonical dated label (DATA_IN_GCS §7.1)
354         ("GX020094.rallies.csv", "GX020094"), # legacy bare label -> SAME clip stem
355         (
356             "GX020094_proxy.rallies.csv",
357             "GX020094_proxy",
358         ), # _proxy is part of the name, kept
359         (
360             "2026-06-21_mahadevpura_1.rallies.csv",
361             "mahadevpura_1",
362         ), # underscores in <name> survive
363         ("v1.csv", "v1"), # bare .csv suffix
364         ("2026-06-22_GX020094.csv", "GX020094"), # dated + bare .csv
365     ],
366 )
367
368 def test_label_clip_name_strips_date_prefix(fname, expected):
369     """A dated label and a bare label must both resolve to the bare videos/<name>

```

```

stem."""
369         assert wmain._label_clip_name(fname) == expected
370
371
372     def test_cmd_train_detector_joins_dated_labels_to_bare_videos(tmp_path, monkeypatch):
373         """Regression (DATA_IN_GCS $7b blocker #1): canonical labels carry an ISO-date
prefix
374         (labels/<date>_<name>.rallies.csv) while videos are bare (videos/<name>.mp4). The
label-derived
375         clip name must strip the date so the video join succeeds ? otherwise every clip is
skipped and
376         train aborts with <2 videos."""
377         import os
378         import subprocess
379

```

4. user (2026-07-06T17:30:44.740Z)

```

1         diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2         index 503f6d8..f96b495 100644
3         --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4         +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5         @@ -22,7 +22,7 @@ import concurrent.futures
6         import logging
7         import os
8         import time
9         -from typing import Any, Dict, List, Optional, Tuple
10        +from typing import Any, Dict, List, Optional, Tuple, Union
11
12        import cv2
13
14        @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15        from backend.pipeline.detectors.base import DetectorRunner
16        from backend.pipeline.segmenters.base import BasePlaySegmenter
17        from backend.providers import provider_registry
18        +
19        +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20        +from backend.results import Failure
21        from backend.sports import sport_registry
22        from backend.utils.paths import output_base
23        from backend.utils.video import extract_video_chunk, get_video_duration
24        @@ -174,10 +177,13 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25            video_path: str,
26            player_pool: Optional[List[str]] = None,
27            max_frames: Optional[int] = None,
28        -    ) -> List[Dict[str, Any]]:
29        -        # override-ignore: the ABC declares the Result contract (Success|Failure)
30        -        # but every live segmenter still returns a bare list ? the documented
31        -        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
32        +    ) -> "Union[List[Dict[str, Any]], Failure]":
33        +        # Partial adoption of the Result contract (the ABC declares Success|Failure).
34        +        # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
35        +        # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused
with a
36        +        # genuine 0-rally video (which still returns a bare ``[]``). Every caller
37        +        # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
38        +        # the bare-list success/empty paths are the documented incremental gap
(CODE_MAP gotchas).
39            label = self.display_label
40            # --- Sport profile + AI provider wiring (identical across segmenters) ---
41            sport_name = self.config.sport
42        @@ -267,7 +273,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

```

```

43         ok, msg = runner.healthcheck()
44         if not ok:
45             logger.error("%s detector environment not ready: %s", label, msg)
46             -         return []
47             +         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
48             +         # failed/retryable instead of writing an empty-but-"successful" result.
49             +         return Failure(
50             +             message=f"{label} detector environment not ready: {msg}",
51             +             is_recoverable=True,
52             +         )
53         logger.info("%s detector env OK: %s", label, msg)
54
55         # --- Phase 2: trajectory -> candidate rally windows ---
56         @@ -278,7 +289,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
57         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
58         if not csv_path:
59             logger.error("%s produced no trajectory; aborting.", label)
60             -         return []
61             +         # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
62             +         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
63             +         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
64             +         # rather than a silent 0-segment "success" (the wf294 incident).
65             +         return Failure(
66             +             message=(
67             +                 f"{label} detector produced no trajectory (timed out or aborted) ?
"
68             +                 "detector failure, not a 0-rally video."
69             +             ),
70             +             is_recoverable=True,
71             +         )
72
73         points = runner.parse_trajectory_csv(csv_path)
74         logger.info(
75         diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
76         index 461d7ca..88c50f6 100644
77         --- a/backend/worker/__main__.py
78         +++ b/backend/worker/__main__.py
79         @@ -114,6 +114,42 @@ def _write_report_json(
80             )
81
82
83         +def _serialize_and_push_outputs(
84         +     scratch: str,
85         +     out_uri: str,
86         +     video_id: str,
87         +     segments: List[dict],
88         +     status: str,
89         +     storage_cfg: dict,
90         +     video_uri: str = "",
91         + ) -> List[str]:
92         +     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
93         +     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
94         +
95         +     Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
96         +     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
97         +     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
98         +     report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to

```

```

99     +     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. """
100    +     out_local = os.path.join(scratch, "outputs", video_id)
101    +     os.makedirs(out_local, exist_ok=True)
102    +     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
103    +     _write_report_json(
104    +         video_id,
105    +         segments,
106    +         status,
107    +         os.path.join(out_local, "report.json"),
108    +         video_uri=video_uri,
109    +     )
110    +     out_prefix = out_uri.rstrip("/")
111    +     pushed: List[str] = []
112    +     for fn in sorted(os.listdir(out_local)):
113    +         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
114    +         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
115    +         pushed.append(uri)
116    +     return pushed
117    +
118    +
119    def _run_startup_checks(config: Any) -> bool:
120        """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
121        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
122        @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
123        )
124
125        def _fail(rc: int) -> int:
126            -         # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
127            -         # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
128            -         # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
129            +         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
130            +         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
131            +         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
132            +         # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
133            +         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
134            +         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
135            +         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
136            +         try:
137            +             _serialize_and_push_outputs(
138            +                 scratch,
139            +                 args.out_uri,
140            +                 video_id,
141            +                 [],
142            +                 "failed",
143            +                 storage_cfg,
144            +                 str(getattr(args, "video_uri", "") or ""),
145            +             )
146            +         except Exception as e: # never mask the real failure on an artifact-push
hiccup
147            +             logger.warning("[worker] could not push failure artifacts: %s", e)
148            +         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
149            +         if getattr(args, "done_uri", None):
150            +             _write_done_marker(

```

```

151         args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
152     @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
153         )
154
155         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
156         out_local = os.path.join(scratch, "outputs", video_id)
157         os.makedirs(out_local, exist_ok=True)
158         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
159         _write_report_json(
160             pushed = _serialize_and_push_outputs(
161                 scratch,
162                 args.out_uri,
163                 video_id,
164                 segments,
165                 status,
166                 os.path.join(out_local, "report.json"),
167                 video_uri=str(getattr(args, "video_uri", "") or ""),
168                 storage_cfg,
169                 str(getattr(args, "video_uri", "") or ""),
170             )
171
172         out_prefix = args.out_uri.rstrip("/")
173         pushed = []
174         for fn in sorted(os.listdir(out_local)):
175             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
176             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
177             pushed.append(uri)
178
179         print(
180             f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
181         )
182     diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
183     index 64909e7..c5ae442 100644
184     --- a/tests/test_tracknet_hybrid.py
185     +++ b/tests/test_tracknet_hybrid.py
186     @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
187         from unittest.mock import MagicMock, patch
188
189         from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
190         +from backend.results import Failure
191
192
193         def _window(start=1.0, end=4.0):
194     @@ -124,7 +125,10 @@ def test_pipeline_healthcheck_failure_aborts(
195
196         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
197         res = seg.process_video("v1", "clip.mp4")
198         - assert res == []
199         + # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
200         + # instead of writing an empty "success".
201         + assert isinstance(res, Failure)
202         + assert "environment not ready" in res.message
203         + # Should bail before ever running inference.
204         + mock_runner_cls.return_value.run_predict.assert_not_called()
205
206     @@ -144,7 +148,36 @@ def test_pipeline_no_trajectory_aborts(
207         mock_provider_registry.get.return_value = MagicMock()
208
209         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
210         - assert seg.process_video("v1", "clip.mp4") == []
211         + res = seg.process_video("v1", "clip.mp4")
212         + # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a

```

```

0-rally [] ?
213 + # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
214 + assert isinstance(res, Failure)
215 + assert "no trajectory" in res.message and res.is_recoverable
216 +
217 +
218 +@_patched
219 +def test_pipeline_zero_windows_is_legit_empty_not_failure(
220 +     mock_runner_cls,
221 +     mock_probe,
222 +     mock_dur,
223 +     mock_extract,
224 +     mock_provider_registry,
225 +     mock_env,
226 + ):
227 +     """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
228 +     parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
229 +     (0-rally "success"), NOT a Failure."""
230 +     mock_env.return_value = "dummy_key"
231 +     mock_dur.return_value = 30.0
232 +     mock_extract.return_value = True
233 +     mock_runner_cls.return_value = _make_runner(windows=[]) # trajectory ok, zero
windows
234 +     mock_provider_registry.get.return_value = MagicMock()
235 +
236 +     seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
237 +     res = seg.process_video("v1", "clip.mp4")
238 +     assert res == [] and not isinstance(res, Failure)
239 +     # The detector DID run (a genuine empty, not an abort).
240 +     mock_runner_cls.return_value.run_predict.assert_called_once()
241
242
243     @_patched
244     diff --git a/tests/test_worker.py b/tests/test_worker.py
245     index 96edbee..63cc18e 100644
246     --- a/tests/test_worker.py
247     +++ b/tests/test_worker.py
248     @@ -10,6 +10,7 @@ import json
249     import pytest
250
251     from backend.config import load_config
252     +from backend.results import Failure
253     from backend.worker import __main__ as wmain
254
255     # ----- pure helpers
256     @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
257
258
259     class _EmptySeg:
260     -     """A segmenter that finds nothing ? the cold-start failure mode."""
261     +     """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""
262
263         def __init__(self, db, config):
264             pass
265     @@ -200,6 +201,20 @@ class _EmptySeg:
266         return []
267
268
269     +class _FailSeg:
270     +     """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
271     +     trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""

```

```

272 +
273 + def __init__(self, db, config):
274 +     pass
275 +
276 + def process_video(self, video_id, path, max_frames=None):
277 +     return Failure(
278 +         message="WASB detector produced no trajectory (timed out or aborted)",
279 +         is_recoverable=True,
280 +     )
281 +
282 +
283     def test_setup_logging_levels():
284         import logging
285
286     @@ -246,12 +261,56 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
287         assert "0 segments" in msgs and "vidZ" in msgs
288         assert "detector_impl=native" in msgs # surfaces the resolved detector
289         assert "native runner UNAVAILABLE" in msgs # points at the likely cause
290     - assert (
291     -         json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
292     -             "segment_count"
293     +         # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
294     +         # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
295     +         report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
296     +         assert report["segment_count"] == 0 and report["status"] == "success"
297     +
298     +
299     +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
300     +     tmp_path, monkeypatch
301     +):
302     +     """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
303     +     as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
304     +     status="failed" so the control plane / alpha-user surface can show an error /
retry."""
305     +     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
306     +     monkeypatch.setattr(
307     +         wmain.validator_registry,
308     +         "run_all_with_context",
309     +         lambda path, cfg: ([_OKResult()], {}),
310     +     )
311     +     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
312     +     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
313     +     video = tmp_path / "in.mp4"
314     +     video.write_bytes(b"FAKE")
315     +     out = tmp_path / "bucket"
316     +     done = tmp_path / "bucket" / "_dispatch" / "tok.done"
317     +
318     +     rc = wmain.main(
319     +         [
320     +             "infer",
321     +             "--video-uri",
322     +             str(video),
323     +             "--out-uri",
324     +             str(out),
325     +             "--scratch",
326     +             str(tmp_path / "s"),
327     +             "--segmenter",
328     +             "wasb_hybrid",
329     +             "--done-uri",
330     +             str(done),

```

```

331         ]
332     -     == 0
333     )
334     +     # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
335     +     assert rc == 3
336     +     # report.json is written as an explicit FAILURE, not a false empty "success".
337     +     report = json.loads(out / "outputs" / "vidX" / "report.json").read_text()
338     +     assert report["status"] == "failed" and report["segment_count"] == 0
339     +     assert report["segments"] == []
340     +     # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
341     +     marker = json.loads(done.read_text())
342     +     assert marker["returncode"] == 3 and marker["status"] == "failed"
343     +     assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
344
345
346     def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
347

```

5. assistant (2026-07-06T17:30:49.596Z)

The finding is about the test at line 316 (now line 346 after the diff),
`test\_cmd\_infer\_validation\_failure\_returns\_2`, asserting only `rc == 2`. Let me verify the test
as it exists in the actual file.

The test I read (lines 316-341 in the current file) confirms:

- Line 341: `assert rc == 2` is the ONLY assertion
- No reading of `report.json`, no status/segment_count assertions
- No `--done-uri` passed, so no marker verification

Let me confirm the `\_fail(2)` path does write the report for the validation branch, and check
the validation failure rc code.

6. user (2026-07-06T17:30:51.341Z)

```

273     )
274
275     def _fail(rc: int) -> int:
276         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
277         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
278         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
279         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
280         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
281         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
282         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
283         try:
284             _serialize_and_push_outputs(
285                 scratch,
286                 args.out_uri,
287                 video_id,
288                 [],
289                 "failed",
290                 storage_cfg,
291                 str(getattr(args, "video_uri", "") or ""),
292             )
293         except Exception as e: # never mask the real failure on an artifact-push hiccup
294             logger.warning("[worker] could not push failure artifacts: %s", e)
295         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).

```

```

296         if getattr(args, "done_uri", None):
297             _write_done_marker(
298                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
299             )
300         return rc
301
302     segments: List[dict] = []
303     segmenter_actual = None
304     segmentation_ran = False
305     status = "failed"
306     try:
307         if failures:
308             print(
309                 "[worker] validation FAILED: "
310                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
311             )
312             return _fail(2)
313         seg_name = args.segmenter or config.indexing.default_segmenter
314         segmenter_cls = segmenter_registry.get(seg_name)
315         if not segmenter_cls:
316             print(
317                 f"[worker] unknown segmenter: {seg_name!r} "
318                 f"(have {list(segmenter_registry.list_available())})"
319             )
320             return _fail(2)
321         segmenter_actual = seg_name
322         result = segmenter_cls(db, config).process_video(
323             video_id, local_video, max_frames=args.max_frames
324         )
325         segmentation_ran = True
326         if isinstance(result, Failure):
327             print(f"[worker] segmenter FAILED: {result.message}")
328             return _fail(3)
329         segments = result.value if hasattr(result, "value") else result
330         status = "success"
331         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
333         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334         if not segments:
335             detector_impl = (
336                 os.environ.get("RALLY_DETECTOR_IMPL")
337                 or getattr(config.indexing, "detector_impl", None)
338                 or "auto"
339             )
340             logger.warning(
341                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343                 "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
344                 "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
345                 "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
346                 "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
347                 "detections, or the input resolution differs from the tuned proxy
resolution. "
348                 "Re-run with -v for the native command + frame counts.",
349                 video_id,
350                 segmenter_actual,
351                 detector_impl,

```

```

352         )
353     finally:
354         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
355         emit_indexer_report(
356             db=db,
357             video_id=video_id,
358             video_path=local_video,
359             config=config,
360             val_results=val_results,
361             val_context=val_context,
362             segmenter_requested=args.segmenter,
363             segmenter_actual=segmenter_actual,
364             was_reingest=False,
365             segmentation_ran=segmentation_ran,
366         )
367
368     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369     pushed = _serialize_and_push_outputs(
370         scratch,
371         args.out_uri,
372         video_id,
373         segments,
374         status,
375         storage_cfg,
376         str(getattr(args, "video_uri", "") or ""),
377     )
378
379     print(
380         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
381     )
382     for u in pushed:
383         print("    ", u)
384
385     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
386     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
387     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
388     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
389     if getattr(args, "done_uri", None):
390         _write_done_marker(
391             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
392         )

```

7. assistant (2026-07-06T17:31:02.768Z)

The finding is fully confirmed by the actual code:

1. ****`\_fail(2)` writes+pushes report.json for validation failures**** ? line 312 (``return _fail(2)``) calls ``_fail``, which at lines 283-292 unconditionally calls ``_serialize_and_push_outputs(..., [], "failed", ...)``, writing a ``status="failed"``, ``segment_count=0`` report.json for the validation branch. This is the newly-added path.
2. ****The validation test asserts only `rc == 2`**** ? ``test_cmd_infer_validation_failure_returns_2`` (test_worker.py lines 316-341) has exactly one assertion, ``assert rc == 2`` (line 341). It reads nothing from ``report.json``, checks no status/segment_count, and passes no ``--done-uri``.
3. ****The rc-3 detector-failure test IS the only one that locks the report**** ? lines 306-313 read report.json and assert ``status=="failed"``, ``segment_count==0``, ``segments==[]``, plus the marker verdict. Nothing equivalent exists for the rc-2 validation branch.

So the newly-added validation-failure report artifact (``status="failed"`, `segment_count==0``) is genuinely unlocked by any assertion. The failure scenario is concrete and reproducible: a refactor mis-passing the status literal for the validation ``_fail`` path (e.g. writing ``"success"`,` or reordering so validation skips ``_serialize_and_push_outputs``) would leave the suite green because the only validation test asserts rc alone.

This is an accurate, verifiable test-coverage gap. Severity is correctly low: it is a coverage/discipline gap, not a defect in shipped behavior ? the production code path is exercised (rc verified) and the same ``_fail`` mechanism is asserted for the rc-3 sibling, so the underlying report-writing is not entirely untested, only the validation-branch specialization is.

8. user (2026-07-06T17:31:10.367Z)

Structured output provided successfully